

System Configuration Implementation

Overview

The system configuration module has been successfully implemented to allow configuration of business rules as specified in the requirements. This module provides a comprehensive solution for managing all system configurations through a unified API.

Requirements Implemented

1. Configure pricing rules and discounts

- **Model:** `PricingRule` with conditions, actions, and priority
- **Service:** `ConfigurationService.createPricingRule()`, `getApplicablePricingRules()`, `calculatePriceWithRules()`
- **API Endpoints:**
 - `POST /api/configuration/pricing-rules` - Create pricing rule
 - `GET /api/configuration/pricing-rules/active` - Get active pricing rules
 - `GET /api/configuration/pricing-rules/active?customerType=...&productCategory=...` - Get applicable rules

2. Set inventory thresholds and alerts

- **Model:** `InventoryThreshold` with critical/low/high thresholds and alert settings
- **Service:** `ConfigurationService.createInventoryThreshold()`, `getInventoryThresholdsForMonitoring()`, `checkInventoryThresholds()`
- **API Endpoints:**
 - `POST /api/configuration/inventory-thresholds` - Create inventory threshold
 - `GET /api/configuration/inventory-thresholds/active` - Get active thresholds

3. Define user roles and permissions

- **Model:** `UserRole` with permissions array and access controls
- **Service:** `ConfigurationService.createUserRole()`, `getUserRoles()`
- **API Endpoints:**
 - `POST /api/configuration/user-roles` - Create user role
 - `GET /api/configuration/user-roles/active` - Get active user roles

4. Configure notification templates

- **Model:** `NotificationTemplate` for email, SMS, in-app, and push notifications
- **Service:** `ConfigurationService.createNotificationTemplate()`, `getNotificationTemplatesByType()`
- **API Endpoints:**
 - `POST /api/configuration/notification-templates` - Create notification template

5. Set up tax rates and shipping costs

- **Model:** `TaxRate` with effective dates and geographic targeting
- **Model:** `ShippingCost` with calculation methods and delivery times
- **Service:** `ConfigurationService.createTaxRate(), getActiveTaxRates(), createShippingCost(), getShippingCostsForCalculation()`
- **API Endpoints:**
 - `POST /api/configuration/tax-rates` - Create tax rate
 - `POST /api/configuration/shipping-costs` - Create shipping cost configuration

6. Manage product categories and attributes

- **Model:** `ProductCategoryConfig` with hierarchy and attributes
- **Model:** `ProductAttribute` with validation rules and options
- **Service:** `ConfigurationService.createProductCategoryConfig(), getProductCategoriesHierarchy(), createProductAttribute(), getProductAttributesByType()`
- **API Endpoints:**
 - `POST /api/configuration/product-categories` - Create product category
 - `POST /api/configuration/product-attributes` - Create product attribute

7. General Settings

- **Model:** `GeneralSettings` for company information, contact details, business hours, etc.
- **Service:** `ConfigurationService.createOrUpdateGeneralSettings(), getGeneralSettings()`
- **API Endpoints:**
 - `POST /api/configuration/general-settings` - Create/update general settings
 - `GET /api/configuration/general-settings/current` - Get current general settings

Architecture

Models

Located in `chicken-processing-backend/src/models/SystemConfiguration.ts`:

- `SystemConfiguration` - Base configuration model with versioning, status, and audit fields
- Configuration types: `PRICING_RULES`, `INVENTORY_THRESHOLDS`, `USER_ROLES`, `NOTIFICATION_TEMPLATES`, `TAX_RATES`, `SHIPPING_COSTS`, `PRODUCT_CATEGORIES`, `PRODUCT_ATTRIBUTES`, `GENERAL_SETTINGS`
- Status types: `ACTIVE`, `INACTIVE`, `ARCHIVED`, `DRAFT`

Services

Located in `chicken-processing-backend/src/services/configurationService.ts`:

- `ConfigurationService` - Comprehensive service with CRUD operations and business logic
- Supports memory store for development (when Firestore is disabled)
- Includes advanced filtering, searching, and pagination

API Routes

Located in `chicken-processing-backend/src/routes/configuration.ts`:

- Full REST API with CRUD operations
- Specialized endpoints for each configuration type
- Authentication middleware integration
- Memory store support for development

Integration

- Registered in main app: `chicken-processing-backend/src/app.ts`
- Available at `/api/configuration` endpoint
- CORS configured for frontend access

Key Features

1. Versioning & Audit

- Automatic version increment on updates
- Created/updated timestamps
- User tracking (createdBy, lastUpdatedBy)

2. Effective Date Management

- `effectiveFrom` and `effectiveTo` dates for time-based configurations
- Automatic filtering of expired configurations

3. Advanced Search & Filtering

- Type, status, and active status filtering
- Text search across name and description
- Sorting by name, date, type
- Pagination support

4. Business Logic

- **Pricing Rules:** Complex condition matching (customer type, product category, quantity, date/time, days of week)
- **Inventory Thresholds:** Multi-level alerts (criticalLow, low, high, criticalHigh)
- **Tax Rates:** Geographic targeting and effective date ranges
- **Shipping Costs:** Multiple calculation methods (flat rate, weight-based, distance-based)

5. Development Support

- Memory store for development when Firestore is disabled
- Sample configurations seeded for testing
- No authentication required in development mode

Testing

Test Script

Located at `chicken-processing-backend/test-configuration.js`:

- Comprehensive API test covering all endpoints
- 14 test scenarios including CRUD operations
- Error handling and validation testing

To Run Tests:

```
cd chicken-processing-backend
# Start the server
npm run dev

# In another terminal
node test-configuration.js
```

Usage Examples

Create a Pricing Rule

```
POST /api/configuration/pricing-rules
{
  "name": "Bulk Discount - Restaurants",
  "description": "10% discount for restaurants ordering more than 50kg",
  "ruleType": "volume_discount",
  "conditions": {
    "customerType": ["restaurant"],
    "minimumQuantity": 50,
    "productCategory": ["fresh_chicken"]
  },
  "actions": {
    "discountPercentage": 10
  },
  "priority": 1,
  "isActive": true
}
```

Create Inventory Threshold

```
POST /api/configuration/inventory-thresholds
{
  "productCategory": "fresh_chicken",
  "thresholds": {
```

```
"criticalLow": 10,  
"low": 20,  
"high": 100,  
"criticalHigh": 150  
},  
"alertSettings": {  
  "emailRecipients": ["manager@bohlokofamily.com"],  
  "notificationChannels": ["email", "in_app"],  
  "alertFrequency": "immediate"  
},  
"isActive": true  
}
```

Get Applicable Pricing Rules

```
GET /api/configuration/pricing-rules/active?  
customerType=restaurant&productCategory=fresh_chicken&quantity=75
```

Next Steps

Frontend Implementation

Create React components for:

1. Configuration dashboard
2. Pricing rule management interface
3. Inventory threshold configuration
4. User role editor
5. General settings page

Integration Points

1. Integrate pricing rules with order processing
2. Connect inventory thresholds with inventory service alerts
3. Apply user roles to authentication middleware
4. Use notification templates in notification service
5. Apply tax rates and shipping costs in checkout

Enhancements

1. Configuration import/export
2. Configuration change history
3. Bulk operations
4. Configuration templates
5. Advanced rule builder UI

Files Created/Modified

New Files

1. `chicken-processing-backend/src/models/SystemConfiguration.ts` - All configuration models
2. `chicken-processing-backend/src/services/configurationService.ts` - Configuration service
3. `chicken-processing-backend/src/routes/configuration.ts` - API routes
4. `chicken-processing-backend/test-configuration.js` - Test script
5. `SYSTEM_CONFIGURATION_IMPLEMENTATION.md` - This documentation

Modified Files

1. `chicken-processing-backend/src/app.ts` - Added configuration routes

Conclusion

The system configuration module provides a robust, flexible foundation for managing all business rules and system settings. It follows SOLID principles, supports both Firestore and memory storage, and includes comprehensive API endpoints for all configuration needs. The implementation is ready for integration with the frontend and other system components.